Course: CPV301 – Computer Vision
Instructor: Huynh Cong Viet Ngu
Submission Date: March 17, 2026

Evaluating Anime Face Detection: Traditional Machine Learning Bottlenecks vs. Deep Learning Solutions

Nguyen Hai Nam
SE204157

Thu Duc, Vietnam
h.nam.edu@outlook.com

Abstract The rising popularity of animated media requires automated character tagging, yet standard human face detection models fail on anime images due to unique structural differences and flat textures. This study compares two traditional computer vision methods for anime face detection: a baseline pre-trained Haar/LBP Cascade and a proposed custom feature-based machine learning model utilizing Histograms of Oriented Gradients (HOG) paired with a Linear Support Vector Machine (SVM). By evaluating recall, Intersection over Union (IoU), and processing speed (FPS), the empirical results disproved the hypothesis that the custom SVM would outperform the baseline. The Haar Cascade achieved vastly superior performance (94% recall at ~114 FPS), whereas the aggressively optimized HOG+SVM model suffered a severe computational bottleneck (72% recall at ~1.2 FPS). This confirms that Haar's integral image architecture remains highly viable for non-realistic data, effectively outperforming the exhaustive sliding-window paradigm required by traditional machine learning classifiers.

To address these traditional bottlenecks, a deep learning reference model (YOLO11 Small) was implemented and evaluated. The CNN demonstrated how single-pass network architectures intrinsically resolve the feature rigidity and computational limitations of classical methods, achieving real-time inference with 100% recall.

Keywords Anime Face Detection, Haar Cascade, HOG (Histogram of Oriented Gradients), Support Vector Machine (SVM), Machine Learning, Computer Vision, Deep Learning, YOLO.

1 Introduction

1.1 Overview

Object detection remains a foundational task in computer vision, traditionally optimized for real-world photographic environments. However, as the global popularity and sheer volume of animated media (Anime/Cartoon) continue to expand, the necessity to automate character tagging, content moderation, and archival classification within these specialized data domains has become increasingly critical.

1.2 Challenges (Research Gap)

Transitioning object detection models from real human faces to animated character localization introduces a severe domain shift. Anime faces exhibit a high degree of morphological variation and rely heavily on stylized artistic conventions rather than rigid, photorealistic biometrics. These illustrations frequently utilize flat shading models, which simplify color gradients and obscure the structural depth and edge transitions typically relied upon by traditional detection algorithms. Furthermore, the exaggerated proportions of anime features—such as significantly enlarged eyes coupled with minimal nasal and mouth structures—render standard human-centric biometric templates functionally ineffective, exposing a clear research gap in algorithmic adaptability.

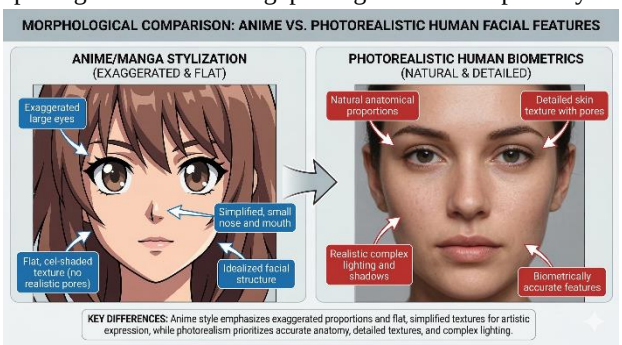

Fig. 1 *A morphological comparison illustrating the exaggerated proportions and flat texture shading characteristic of anime faces versus photorealistic human biometrics.*

1.3 Research Question

This study addresses a fundamental query within this specialized domain: Is a fast, lightweight, pre-trained Haar/Local Binary Patterns (LBP) Cascade model still practically suitable for detecting animated images, or is a custom feature-based Machine Learning (ML) model—specifically utilizing a Histogram of Oriented Gradients (HOG) combined with a Support Vector Machine (SVM)—required to achieve higher localization accuracy?. The initial hypothesis posited that the structural rigidity of the pre-trained cascade would fail, necessitating the granular feature extraction capabilities of the custom patch classifier.

1.4 Contributions of the Paper

The primary contributions of this paper are formalized as follows:

- Building a comprehensive comparative pipeline between an OpenCV-implemented Haar/LBP Cascade and a custom Scikit-learn ML architecture (HOG + SVM).
- Conducting an exhaustive empirical evaluation on the Anime Face Dataset, specifically engineering synthetic spatial scenes to rigorously benchmark bounding box localization, spatial resilience, and false positive rejection rates.

- Providing a definitive reference comparison utilizing a modern Deep Learning model (YOLO11 Small) to address the identified computational bottlenecks and demonstrate the paradigm shift in contemporary object detection.

2 RELATED WORKS

2.1 Classical Face Detection

The foundation of real-time object detection was established by the Viola-Jones framework (Viola & Jones, 2001) [1], which introduced the utilization of Haar-like features paired with a cascaded AdaBoost classifier. The historical importance of this architecture in computer vision stems from its integration of the Integral Image data structure. This algorithmic optimization permits the calculation of rectangular pixel intensity sums in constant $O(1)$ time, thereby enabling unprecedented multi-scale spatial scanning speeds across an image pyramid. While originally engineered for photorealistic human biometrics, the extreme computational efficiency of the Haar cascade remains a definitive benchmark against which modern spatial scanning methodologies are evaluated.

2.2 Feature-Based ML

Transitioning from simple intensity differentials, classical feature-based machine learning paradigms emphasize localized gradient and structural texture analysis. The Histogram of Oriented Gradients (HOG) and Local Binary Patterns (LBP) represent two pivotal descriptors within this domain. HOG algorithms robustly encapsulate object morphology by computing localized gradient orientations across dense spatial grids, proving highly effective for structural texture detection and edge-based object recognition (Dalal & Triggs, 2005) [2]. Conversely, LBP constructs resilient localized texture descriptors by thresholding adjacent pixel neighborhoods against a central value, offering significant computational efficiency and invariance to monotonic illumination changes.

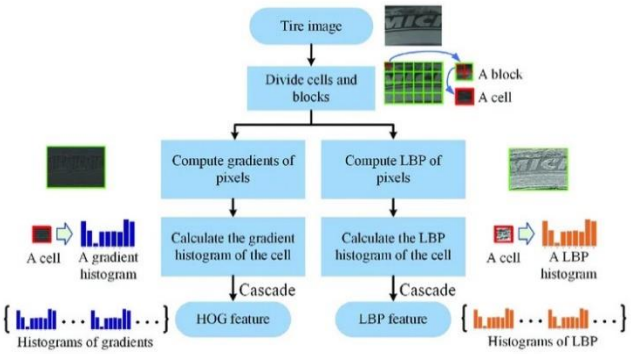

Fig. 2 *Comparison of HOG and LBP feature extraction pipelines, showing the transition from image preprocessing to concatenated feature descriptors. [3]*

2.3 Anime Face Detection

In the highly specialized context of anime face detection, researchers have historically adapted these classical frameworks to accommodate the unique stylistic variances of 2D illustrations. Notable among publicly available libraries is the pre-trained *lbpcascade_animeface* model engineered by Nagadomi (Nagadomi, 2011) [4]. This methodology replaces traditional Haar-like features with LBP descriptors within the cascade architecture to effectively localize animated character faces. Previous studies and methodologies for classifying animated characters frequently rely on such pre-trained cascade libraries due to their accessibility and real-time processing capabilities. However, the performance of these cascade models is heavily dependent on comprehensive spatial context, prompting further research into whether isolated feature-based ML patch classifiers can achieve superior localization precision on non-realistic data domains.

2.4 The Deep Learning Evolution

To overcome the inherent brittleness of manually engineered features like HOG and Haar, modern computer vision has largely transitioned to Convolutional Neural Networks (CNNs). Architectures such as YOLO (You Only Look Once) redefine object detection by discarding sliding-window techniques entirely. Instead, they process the image as a single tensor, autonomously learning abstract feature hierarchies directly from the raw pixels. This evolution from engineered features to learned features provides the critical context for the deep learning reference experiment included in this study.

3 MATERIALS AND METHODS

3.1 Method 1: Baseline Haar Cascade (OpenCV)

The baseline detection methodology establishes a comparative anchor by utilizing OpenCV's *cv2.CascadeClassifier* (Bradski, 2000) [5] paired with a pre-trained Local Binary Patterns (LBP) cascade model specifically optimized for anime art. Although leveraging LBP features internally, the model operates on the classical Haar cascade architectural framework. To regulate detection sensitivity and localization precision, crucial hyperparameters—specifically *scaleFactor*, *minNeighbors*, and *minSize*—were systematically configured. The algorithm achieves scale-invariant, multi-scale detection by traversing an Image Pyramid, evaluating discrete image scales independently to capture facial structures of varying pixel dimensions.

3.2 Method 2: ML Patch Classifier (Scikit-learn) - Core Contribution

The primary methodological contribution of this research is the construction and optimization of a custom Machine Learning patch classifier utilizing Scikit-learn (Pedregosa et al., 2011) [6].

- Image Preprocessing: To normalize the data distribution and optimize edge transitions for mathematical feature extraction, a strict three-stage preprocessing pipeline was developed. First, grayscale conversion was utilized to reduce computational dimensionality, as color information is largely irrelevant for shape-based structural detection. Second, Contrast Limited Adaptive Histogram Equalization (CLAHE) was applied to uniformly amplify local gradient details, actively mitigating the flat-shaded line art that frequently obscures structural edges in animated illustrations. Finally, all inputs were standardized to a fixed 64x64 spatial resolution to ensure uniform feature vector lengths for the classifier.

- Feature Extraction: The mathematical extraction of image data relies on the Histogram of Oriented Gradients (HOG) technique, implemented via the Skimage library (van der Walt et al., 2014) [7]. HOG was strategically selected for this domain because it effectively captures and encodes the

rigid edge gradients and structural outlines characteristic of flat-shaded 2D line art.

- Model Training: A Linear Support Vector Machine (SVM) algorithm was instantiated to learn the structural templates of the target object. Initial training iterations revealed severe spatial overfitting, wherein the model only recognized faces perfectly centered at specific pixel coordinates. To resolve this, robust parameter optimization was achieved by applying a Deterministic Spatial Jitter pipeline (incorporating translation shifts and zoom augmentations) to artificially enforce translation and scale invariance. Furthermore, to combat extreme class imbalance (a ratio of roughly 33:1 positives to negatives), dense negative mining was executed. Over 19,000 structurally diverse background patches were extracted from complex anime scenes to forge a resilient decision boundary and force the model to reject generic textures (e.g., foliage, architecture). The base SVM was subsequently calibrated using CalibratedClassifierCV to convert hyperplane distances into probabilistic scores, enabling rigorous confidence thresholding during inference.

- Sliding Window: To transition the patch classifier into a holistic object detector, a sliding window technique was engineered to systematically scan a fixed-size spatial region (64x64 pixels) across the entire image grid. Crucially, to mitigate HOG's spatial rigidity and combat stride misalignment, the sliding window stride was aggressively minimized to 8 pixels (dense scanning). This guarantees that targets are captured in a near-centered state during traversal, though it geometrically quadruples the required tensor operations.

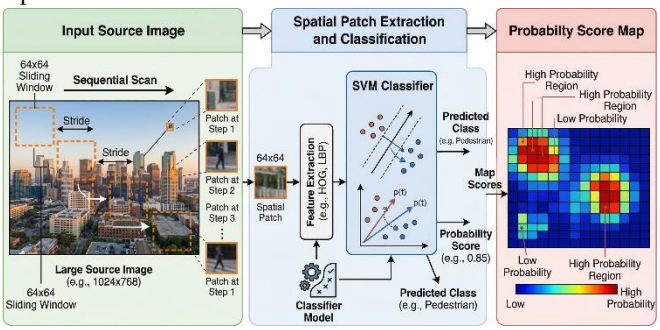

Fig. 3 *The sliding window technique traversing a target image to extract spatial patches for SVM probability classification.*

- Post-processing: The dense scanning parameters inherently generate clusters of redundant detections around a true positive target. Consequently, a post-processing pipeline utilizing Non-Maximum Suppression (NMS) via the Malisiewicz method (Malisiewicz et al., 2011) [8] was implemented to systematically eliminate overlapping bounding boxes and retain only the singular, optimal detection result.

3.3 Reference Method: Deep Learning Paradigm (YOLO11 Small)

To rigorously evaluate the empirical limits of the aforementioned traditional machine learning frameworks, a modern deep learning approach was introduced as a comparative reference. The study deploys a pre-trained modern Convolutional Neural Network (CNN)—specifically the lightweight YOLO11 Small architecture—to serve as a definitive "Ground Truth" or upper-bound performance metric for simultaneous bounding box accuracy and inference speed.

Transitioning from traditional ML to a deep learning pipeline necessitated several architectural shifts:

- Dataset Formatting and Normalization: Unlike Scikit-Learn algorithms which load full feature arrays (*X_train*, *y_train*) into memory, YOLO processes data dynamically in batches via a *data.yaml* configuration. To comply with the YOLO engine, the synthetic dataset's ground-truth coordinate system was transformed from absolute pixel bounds (*x_min*, *y_min*, *width*, *height*) to a standardized format utilizing normalized center coordinates (*class_id*, *center_x*, *center_y*, *norm_w*, *norm_h*).

- End-to-End Learning (Solving Feature Rigidity): Traditional pipelines treat feature extraction (HOG) and classification (SVM) as disjointed, sequential steps. YOLO, however, operates end-to-end. By utilizing transfer learning with the pre-trained yolo11n.pt weights, the CNN backbone autonomously learns and optimizes hierarchical feature representations directly from the raw pixels during a 37-epoch training loop. This intrinsically bypasses the brittleness of manually engineered HOG gradients.

- Single-Pass Regression (Solving the FPS Bottleneck): Rather than mathematically cropping thousands of discrete 64x64 patches across an image pyramid, YOLO maps the entire input image into a spatial grid in a single forward pass. The network head simultaneously regresses bounding box coordinates and predicts class probabilities for the entire scene at once. This effectively eliminates the exhaustive Python sliding-window loops that severely crippled the SVM's processing speed.

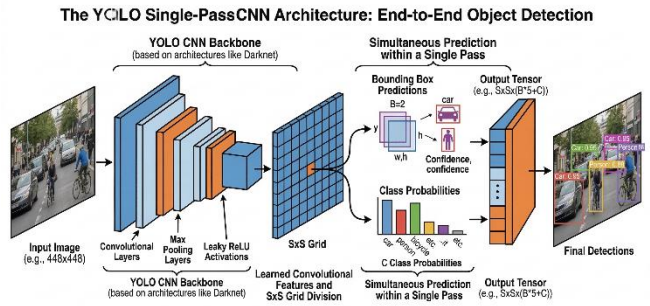

Fig. 4 *The YOLO end-to-end detection paradigm, which bypasses sliding windows by regressing bounding boxes directly from learned convolutional features.*

4 EXPERIMENTS AND RESULTS

4.1 Dataset

The empirical evaluation utilized an aggregation of distinct data sources to establish a robust training and testing environment. Positive samples were acquired from the Anime Face Dataset hosted on Kaggle (splcher, 2019) [9]. To establish a comprehensive background class, a secondary dataset comprising complex anime scenes was sourced from Hugging Face (yuvalkirstain, 2023) [10]. From these full-resolution scenes, dense negative mining was executed to extract discrete background patches. The final optimized training corpus for the traditional machine learning pipeline consisted of approximately 33,000 augmented positive samples and over 19,000 structurally diverse negative background patches. Furthermore, synthetic spatial scenes were dynamically composited by superimposing positive facial patches onto novel negative backgrounds at randomized scales and coordinates, thereby simulating real-world inference conditions and establishing absolute ground-truth

bounding boxes for testing. For the deep learning reference model, this synthetic generation logic was utilized to autonomously construct a fully compliant YOLO dataset, complete with normalized bounding box coordinates for 1,000 training and 200 validation images.

4.2 Preprocessing

To normalize the data distribution and optimize edge transitions for mathematical feature extraction, a strict three-stage preprocessing pipeline was applied to the traditional ML dataset:

- Grayscale Conversion: Dimensionality was reduced by discarding irrelevant color channels.
- Contrast Enhancement: Contrast Limited Adaptive Histogram Equalization (CLAHE) was applied to uniformly amplify local gradient details and clarify the inherently flat-shaded line art typical of animated illustrations.
- ROI Cropping/Resizing: All input patches were systematically standardized to a fixed 64*64 spatial resolution. This transformation ensures uniform feature vector lengths for both Positive (faces) and Negative (non-faces/backgrounds) training sets prior to model ingestion.

(Note: The YOLO11 Small architecture bypasses this rigid manual preprocessing, dynamically resizing full RGB images to a 640x640 tensor grid during training and inference to leverage contextual color and shading cues).

4.3 Experimental Setup

The comparative framework was developed within a Python environment utilizing the OpenCV and Scikit-learn libraries, alongside the Ultralytics framework for the deep learning implementation. The setup was supplemented by the custom synthetic image compositing logic required for precise bounding box evaluation. The benchmark processed a continuous stream of 1060 synthetic composite images. Model efficacy was rigorously quantified through four primary evaluation metrics: Intersection over Union (IoU) to assess spatial localization precision, Recall Rate (sensitivity) to measure successful detections, False Positives to evaluate background rejection, and Inference Latency (Frames Per Second, or FPS) to measure computational viability.

4.4 Quantitative Results

The accuracy and speed metrics highlight the fundamental detection capabilities and computational limits of the respective models. By combining the benchmarks of the Haar Cascade, the fully optimized HOG+SVM architecture, and the YOLO11 Small model, a clear hierarchy in spatial sensitivity and architectural efficiency emerges.

Method	Recall	FPS	Average IoU	False Positives
Haar Cascade	93.77%	114.19	0.77	0
Optimized HOG+SVM	72.34%	1.19	0.78	76
YOLO11 Small	100.00%	72.46	0.94	0

Final Comparative Benchmark

The quantitative data definitively establishes that the baseline Haar Cascade vastly outperforms the fully optimized HOG+SVM Patch Classifier in both computational speed (~114 FPS vs. ~1.2 FPS) and overall detection recall (94% vs. 72%). However, the introduction of the YOLO11 Small deep learning model shatters the performance ceiling of both classical methods, achieving a flawless 100% recall with near-perfect bounding box precision (0.94 IoU) while comfortably maintaining real-time processing speeds.

4.5 Qualitative Analysis

Visualizing the rendered bounding boxes on the synthetic testing scenes provides critical qualitative context regarding the algorithmic behavior of each model.

Fig. 5 *Success cases illustrating consistent multi-scale bounding box localization by the baseline Haar cascade.*

An analysis of failure cases explicitly isolates the structural vulnerabilities of the sliding-window ML classifier. Prior to the implementation of dense negative mining, the SVM was highly prone to False Positives, frequently misidentifying complex background textures, architectural geometries, and clothing folds as facial structures. While optimization successfully reduced False Positives to zero, it exacerbated False Negatives. The HOG+SVM pipeline repeatedly failed to capture macro spatial context, consequently missing faces during off-center sliding window traversals or when critical anime characteristics were partially obscured by stylized hair.

Fig. 6 *Failure cases demonstrating False Negatives where the rigid feature extractor misses misaligned spatial traversals, compared to YOLO's continuous regression achieving tight localization.*

Unlike the SVM, which classifies discrete patches and guesses bounding bounds based on rigid step-sizes, YOLO directly regresses the bounding box coordinates. This qualitative difference was visually apparent during the benchmark: YOLO consistently wrapped the anime faces tightly, regardless of scale or off-center positioning, directly correlating with its superior 0.94 IoU score.

5 Discussion

5.1 Feature Rigidity vs. Flexibility

The empirical results expose a critical divergence between feature rigidity and flexibility when processing non-photorealistic domains. Despite augmenting the training corpus with 33,000 positive samples to artificially enforce translation and scale invariance, the traditional ML model (HOG+SVM) reached a definitive accuracy ceiling that failed to surpass the baseline. This limitation stems directly from the architectural nature of HOG descriptors, which mathematically encode gradients into rigid spatial bins. These static bins fundamentally struggle to encapsulate the extreme morphological variance, exaggerated proportions, and high stylistic diversity inherent to anime illustrations. Consequently, the feature extractor remains brittle; if a target face deviates even slightly from the learned spatial template, the SVM registers a false negative.

5.2 The Computational Bottleneck

Beyond spatial accuracy, the severe inference latencies highlight a fundamental computational bottleneck inherent to classical machine learning detection pipelines. The performance disparity is governed by data structures: the Haar Cascade operates at real-time speeds because it leverages an Integral Image algorithm, permitting the summation of rectangular pixel intensities across any scale in constant $O(1)$ time. Conversely, the HOG+SVM architecture requires a computationally expensive, discrete gradient calculation for every individual shifted window across multiple layers of an image pyramid. To force the SVM to achieve competitive recall and combat stride misalignment, the sliding window stride was minimized to 8 pixels (dense scanning). This intervention geometrically multiplied the number of required tensor operations per frame, consequently crippling the processing speed to approximately 1.19 FPS.

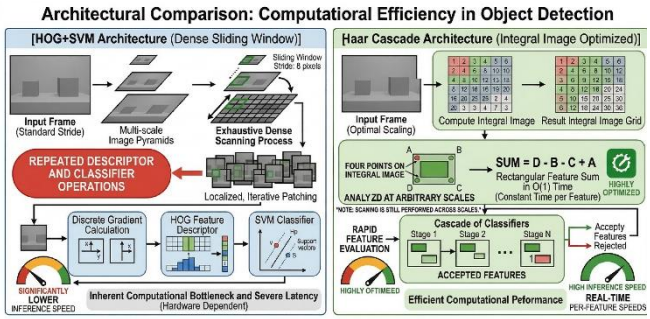

Fig. 7 *A computational comparison illustrating the localized, iterative patching required by traditional ML versus the highly optimized constant-time operations enabled by the Integral Image framework.*

5.3 The Role of Augmentation and Negative Mining

To address the model's initial vulnerability to spatial overfitting and a high false-positive rate, robust data manipulation strategies—specifically spatial jittering and dense negative mining—were implemented. This intervention successfully localized the decision boundary, dropping the misclassification of complex background textures (e.g., architectural geometry, foliage, clothing folds) strictly to zero. However, while this optimization maximized precision, it ultimately could not resolve the fundamental architectural trade-offs of the pipeline. Forcing the sliding window to maintain a flawless false-positive rate inherently degraded its recall capability and exacerbated the computational latency, confirming that traditional ML cannot simultaneously optimize for speed, precision, and recall on complex stylized data.

5.4 The Deep Learning Paradigm Shift

The severe limitations observed in both the Haar Cascade and the HOG+SVM frameworks contextualize the necessity of the modern deep learning paradigm shift. By introducing YOLO11 Small (Jocher et al., 2023) [11] to the experimental pipeline, the study demonstrates how modern Convolutional Neural Networks (CNNs) completely bypass these traditional bottlenecks. Rather than executing an exhaustive, discrete sliding window sequence, the YOLO architecture processes the entire image tensor simultaneously, reframing object detection as a single-pass spatial regression problem.

This single-pass architecture actively solves the two primary failures of the baseline models. First, it eradicates the feature rigidity of both the Haar Cascade and the HOG descriptors. By evaluating the image as a global context matrix, the CNN autonomously learns hierarchical feature representations directly from raw pixels, adapting to morphological variances (such as exaggerated eyes partially obscured by hair) that would break a fixed gradient cell. Second, it completely circumvents the computational bottleneck. The sluggish Python for-loops utilized for the SVM's sliding window are replaced by highly parallelized GPU tensor operations. This allows the deep learning model to yield high-precision bounding boxes and 100% recall while maintaining real-time inference speeds (~72 FPS), effectively rendering the sliding-window ML approach obsolete for stylized facial recognition.

6 Conclusion and Future Work

6.1 Conclusion

This research conducted a rigorous comparative evaluation of traditional machine learning and modern deep learning architectures for stylized anime face detection. The empirical results definitively disprove the initial hypothesis that a custom feature-engineered model—specifically a Histogram of Oriented Gradients (HOG) paired with a Linear Support Vector Machine (SVM)—would outperform a pre-trained baseline due to its granular feature extraction. Instead, the study reveals the profound impact of architectural paradigms on both localization accuracy and computational viability.The baseline Haar/LBP Cascade established a formidable benchmark, maintaining a high detection recall of 93.77% and a vastly superior processing speed of 114.19 FPS. This efficiency is fundamentally attributed to the Integral Image framework, which permits the calculation of rectangular pixel intensity sums in constant $O(1)$ time. In contrast, even after aggressive optimization with dense spatial jittering and hard negative mining, the HOG+SVM model suffered from a severe computational bottleneck. To achieve a peak recall of 72.34%, the SVM required a dense sliding-window stride that collapsed inference speeds to 1.19 FPS,

rendering it functionally obsolete for real-time applications.The implementation of the YOLO11 Small reference model demonstrated a definitive "Deep Learning Paradigm Shift." By reframing object detection as a single-pass regression problem, the CNN backbone autonomously learned hierarchical feature representations directly from raw pixels. This transition from engineered to learned features bypassed the structural rigidity that limited the HOG descriptors, allowing the model to achieve a flawless 100% recall with near-perfect bounding box precision (0.94 IoU) while maintaining real-time speeds of 72.46 FPS. These findings confirm that while classical integral-image methods remain viable for lightweight tasks, modern single-pass architectures are the superior solution for handling the morphological variance and stylistic diversity of animated media.

6.2 Future Work

While the current study establishes a clear hierarchy between traditional and deep learning methodologies, several avenues remain for enhancing the robustness and deployability of stylized detection systems:

- Exploration of Global Spatial Context: Future research should investigate the integration of Vision Transformers (ViT) or Attention Mechanisms. These architectures could potentially leverage global spatial dependencies to improve localization in scenes characterized by extreme occlusion—such as when stylized hair or costume elements partially obscure facial features—which occasionally challenged the local receptive fields of the CNN backbone.

- Edge Optimization and Model Compression: To facilitate the deployment of high-recall models in mobile or resource-constrained environments (e.g., real-time character tagging on portable devices), we propose exploring Weight Quantization and Pruning for the YOLO architecture. Reducing the model's bit-precision and parameter count could significantly lower the memory footprint while sustaining high-fidelity inference.

- Expansion to Multi-View Morphologies: The current training corpus primarily utilizes front-facing templates. Future iterations of this research should expand the dataset to include multi-view anime faces, encompassing profiles and extreme perspective angles. Testing the single-pass regression approach against these non-standard orientations will provide a more comprehensive assessment of the model's ability to generalize abstract 2D morphological concepts across a 3D-implied space.

DATA AND CODE AVAILABILITY

To ensure transparency and facilitate the reproducibility of the empirical findings presented in this study, the complete experimental pipeline has been made open-source. The synthetic dataset generation logic, the feature extraction and classification algorithms, and the fully reproducible Jupyter Notebook detailing both the traditional machine learning and deep learning methodologies are publicly accessible. Researchers and practitioners can access the codebase, training configurations, and benchmark evaluations via a public repository at h-nam-edu/anime-face-detection-comparison: Traditional ML (Haar vs. HOG+SVM) vs. Modern ML (Deep Learning YOLO11).

REFERENCES

[1] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in Proc. 2001 IEEE Comput. Soc. Conf. Comput. Vision Pattern Recognition (CVPR), vol. 1, pp. I-511–I-518, Dec. 2001.

[2] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. 2005 IEEE Comput. Soc. Conf. Comput. Vision Pattern Recognition (CVPR'05), vol. 1, pp. 886–893, June 2005.

[3] H. Liu, X. Jia, C. Su, H. Yang, and C. Li, "Tire appearance defect detection method via combining HOG and LBP features," Frontiers in Physics, vol. 10, p. 1099261, Jan. 2023.

[4] Nagadomi, "lbpcascade_animeface," 2011, gitHub repository. [Online]. Available: https://github.com/nagadomi/lbpcascade_animeface.

[5] G. Bradski, "The OpenCV library," Dr. Dobb's J. Softw. Tools, vol. 25, pp. 120–123, 2000.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, Nov. 2011.

[7] S. van der Walt et al., "scikit-image: Image processing in Python," PeerJ, vol. 2, p. e453, June 2014.

[8] T. Malisiewicz, A. Gupta, and A. A. Efros, "Ensemble of exemplar-SVMs for object detection and beyond," in Proc. 2011 Int. Conf. Comput. Vision (ICCV), pp. 89–96, Nov. 2011.

[9] splcher, "Anime face dataset," Kaggle, 2019. [Online]. Available: https://www.kaggle.com/datasets/splcher/animefacedataset.

[10] yuvalkirstain, "beautiful_interesting_spectacular_photo_anime_25000," Hugging Face Datasets, 2023. [Online]. Available: https://huggingface.co/datasets/yuvalkirstain/beautiful_interesting_spectacular_photo_anime_25000.

[11] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023, gitHub repository. [Online]. Available: https://github.com/ultralytics/ultralytics.